Jose Alonso

CS-405

Secure Coding and Security Policies Reflection

Throughout this course, I learned that software security needs to be considered from the beginning of development. Before this course, I mainly thought about making program work correctly. Now I understand that a program also needs to protect user information and prevent security problems. My work with Green Pace helped me understand how secure coding standards, risk assessments, zero trust, and security policies can improve software security.

Adoption of Secure Coding Standards

One of the most important lessons I learned is that security should not be left until the end of development. If developers wait until a program is finished looking for security problems, fixing them can take more time and money. Following a secure coding standard helps developers avoid common mistakes from the beginning. For example, Green Pace can use standards for validating input, preventing SQL injection, handling errors, and checking array and container bounds. Tools such as Visual Studio and Cppcheck can help identify potential problems before software is released. I learned that secure coding is not just about finding errors; it is about building good habits that make software safer and more reliable.

Risk Assessment and Cost-Benefit of Mitigation

Risk assessment helps a company decide which security problems need attention first. Developers should consider how likely a problem is to happen and how much damage it could cause. For example, a vulnerability that exposes customer information should receive more attention than a minor issue with little effect on security. Fixing problems early may require extra testing and development time, but it can prevent larger expenses from data breaches, system downtime, and damage to the company's reputation. I learned that security decisions should consider both the cost of fixing a problem and the potential cost of leaving it unresolved. One question I would continue to investigate is how companies can best prioritize risks when they have limited time and resources.

Zero Trust

Zero trust is a security approach based on the idea that access should not automatically be trusted just because a user or device is inside a network. Access should be verified, and users should only receive the permissions they need to do their jobs. For Green Pace, this could include using multifactor authentication, limiting access to sensitive data, and reviewing permissions regularly. These practices can reduce the chance of unauthorized access if an account is compromised. I learned that protecting a system requires more than a firewall or password. Security must continue throughout the system, including when users access data and applications.

Security Policy Implementation and Recommendations

Security policies help developers follow consistent rules instead of making security decisions differently for every project. Green Pace should provide clear secure coding standards, require code reviews and testing, use automated security tools, encrypt sensitive information, and apply appropriate access controls. Employees should also receive training so they understand why these practices matter. Security policies should be reviewed and updated as new threats develop. In my opinion, the best approach is to make security part of the normal development process rather than treating it as a separate task. Overall, this course helped me understand that secure coding, risk management, zero trust, and clear security policies work together to protect software and its users.